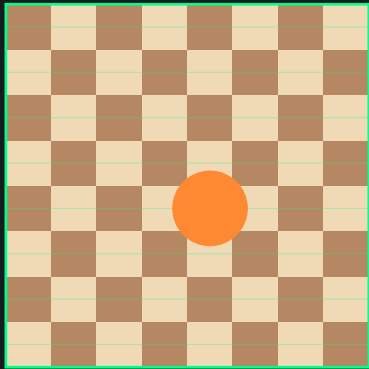


PROJECT BLUEPRINT



Chess AI Agent Extension

Real-time board analysis · Best move suggestions
Post-game deep analysis · AI coaching
Browser extension + Desktop overlay

VISION

TECH STACK

ROADMAP

Prepared by: AI Engineering Team · Version 1.0 · June 2026

TABLE OF CONTENTS

#	Section	Topics
1	Project Vision & Goals	Problem statement, objectives, success metrics
2	System Architecture	Component diagram, data flow, tech layers
3	Core Features	Real-time analysis, move suggestions, post-game
4	Technology Stack	Frontend, backend, AI/ML, infrastructure
5	Screen Capture & Vision	Capture pipeline, board detection, FEN parsing
6	AI Engine & Move Analysis	Stockfish, Claude API, evaluation, reasoning
7	UI/UX Design	Overlay, dashboard, move cards, settings
8	Post-Game Analysis Module	PGN parsing, blunder detection, insights
9	Development Roadmap	5 phases, milestones, deliverables
10	Risks & Mitigations	Technical, security, performance risks
11	API & Integration Points	Endpoints, webhooks, data schemas
12	Deployment & Distribution	Chrome extension, Electron, packaging



Project Vision & Goals

Why we're building this and what success looks like

Problem Statement

Chess players at all levels — from beginners to club players — struggle to improve because they lack real-time guidance during live games. Post-game engines like Lichess and Chess.com offer analysis, but only *after* the damage is done. There is no tool that watches your screen, understands the board position, and instantly explains **why** a move is good or bad in plain language.

Core Objectives

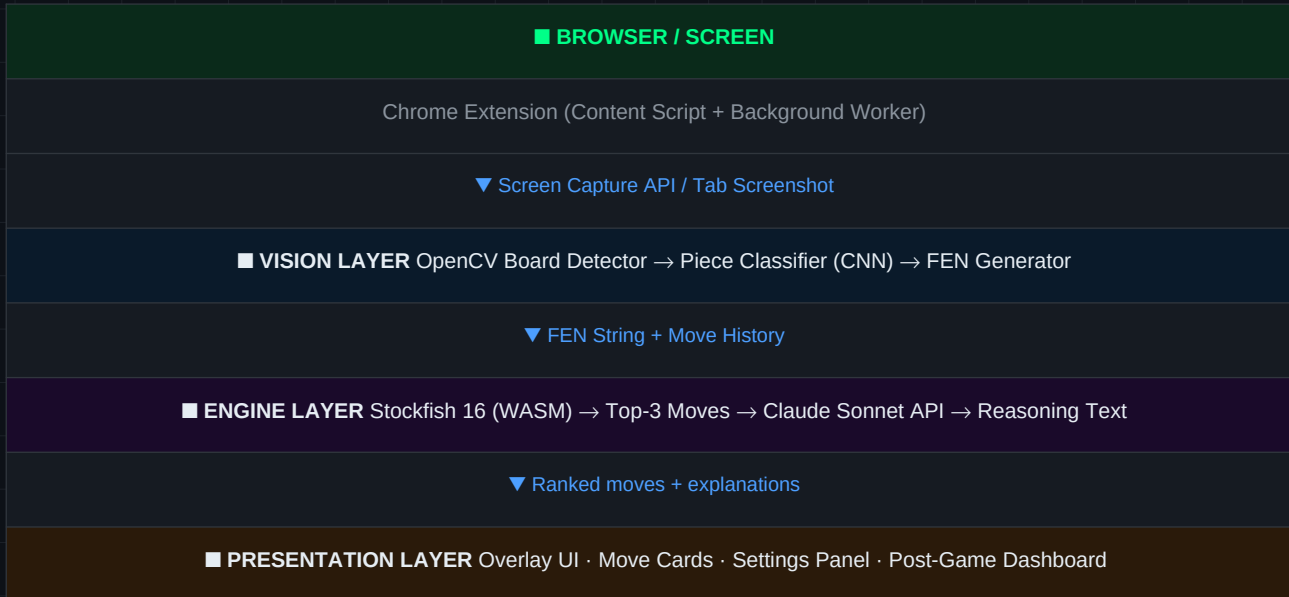
	Live Screen Watching Continuously capture and parse the chessboard from any website or app HIGH
	Best Move Suggestions Rank top-3 moves with Stockfish engine evaluation scores HIGH
	Natural Language Reasons Claude AI explains every move suggestion in plain English HIGH
	Post-Game Deep Dive After the game, replay every move with AI-generated coaching HIGH
	Low Latency (<200ms) Analysis must feel instant — no waiting between moves HIGH

Success Metrics

Metric	Target	Measurement
Board detection accuracy	≥ 99.5%	Automated test suite (500 positions)
Move suggestion latency	< 200 ms	P95 measured in production
Natural language quality	4.5/5 user rating	In-app feedback widget
Post-game insight coverage	100% of moves	Every move has a comment
Extension install size	< 15 MB	Chrome Web Store package
Memory usage (idle)	< 80 MB RAM	Chrome task manager

Architecture Overview

The extension is composed of four major layers: the **Capture Layer** grabs raw screen pixels, the **Vision Layer** detects and parses the board, the **Engine Layer** runs Stockfish analysis and Claude reasoning, and the **Presentation Layer** shows results to the user.



Data Flow

Step	Input	Process	Output	Latency
1	Screen pixels	Tab.captureVisibleTab()	JPEG frame	~5 ms
2	JPEG frame	OpenCV corner detection	Board ROI crop	~15 ms
3	Board ROI	CNN piece classifier	Piece grid (8×8)	~40 ms
4	Piece grid	FEN encoder	FEN string	~1 ms
5	FEN string	Stockfish WASM	Top moves + eval	~60 ms
6	FEN + moves	Claude Sonnet API	Reasoning text	~70 ms
7	Moves + text	Overlay renderer	UI cards shown	~5 ms



Core Features

Detailed breakdown of all product features

Feature 1 — Real-Time Board Monitoring

The agent watches the active browser tab using the Chrome `chrome.tabs.captureVisibleTab()` API, capturing a screenshot every 500ms. A lightweight change-detector compares frames and only triggers full analysis when the board state changes, keeping CPU usage minimal.

- Detects piece moves, en passant, castling, and promotions automatically
- Works on Chess.com, Lichess.org, Chess24, and any image-rendered board
- Configurable polling interval (200ms to 2000ms) via settings

Feature 2 — Move Suggestion Panel

After each detected move, Stockfish evaluates the position to depth 18 and returns the top 3 candidate moves. Each move is sent to Claude Sonnet with the full game context and position evaluation, which returns a 2–3 sentence tactical explanation in plain English.

Move	Eval	Type	AI Explanation (example)
e4–e5 ■	+2.4	Best	Advances the pawn to the center, seizing space and threatening Nf6. This forces Black to respond defensively.
Nf3–d4 ■	+1.8	Good	Knight centralization with an immediate fork threat on c6 and f5. Maintains solid pawn structure.
Bc1–g5 ■	+1.1	Equal	Pins the knight to the queen, but allows Black to break the pin with h6. Consider only if you want quiet play.

Feature 3 — Post-Game Analysis

After the game ends, the agent stores the full PGN (Portable Game Notation) and launches a dedicated analysis dashboard. Every single move is evaluated and annotated with accuracy scores, mistake classifications, and improvement suggestions.

- Move accuracy %: shows your overall accuracy vs opponent
- Blunder / Mistake / Inaccuracy / Best Move classification for every half-move
- Critical moments highlighted — positions where the game turned
- AI coaching summary: "You lost the game at move 23 when you missed Rxd5 — a common tactical pattern called a discovered attack"
- Opening name detection and theory deviation point
- Endgame technique grading

4

Technology Stack

Every tool, library, and service used in the system

■ Frontend / Extension

JS

JavaScript (ES2022)

Core extension logic, content scripts, background worker



React 18

Overlay UI, dashboard panels, move cards



Tailwind CSS

Dark theme utility-first styling for overlay



Vite + CRXJS

Build tooling for Chrome extension with HMR



Chrome Extension MV3

Manifest V3 APIs: tabs, storage, scripting

■ AI & Engine



Stockfish 16 (WASM)

Chess engine compiled to WebAssembly, runs in browser



Claude Sonnet API

Anthropic API for natural language move explanations



chess.js

FEN/PGN parsing, legal move validation, game state



OpenCV.js (WASM)

Computer vision for board detection in browser



TensorFlow.js

CNN piece classifier running locally in browser

■ Desktop / Overlay



Electron 28

Desktop app wrapper for native screen capture



Desktopify

Cross-process window overlay (transparent frameless)



robotjs / screenshot-desktop

System-level screen capture for non-browser boards



IPC Bridge

Electron main ↔ renderer process communication



electron-builder

Cross-platform packaging (.exe, .dmg, .ApplImage)

■ Backend Services



Node.js 20 + Fastify

REST API server for game storage and user accounts



PostgreSQL 15

Persistent storage for games, analysis, user data



Redis

Caching FEN→analysis results, session tokens



Auth0 / JWT

User authentication and authorization



TimescaleDB

Time-series data for move timings and performance trends

■ Infrastructure



Vercel / Railway

Serverless deployment for API and dashboard web



Docker + Compose

Containerized development and staging environment



GitHub Actions

CI/CD pipeline: lint → test → build → publish



Sentry

Error tracking and performance monitoring in production



PostHog

Privacy-friendly product analytics and feature flags

Stage 1 — Screen Capture

The Chrome extension uses `chrome.tabs.captureVisibleTab()` to take a JPEG screenshot of the active tab. A frame differencing algorithm (pixel-level sum of squared differences) determines if the board has changed before triggering expensive vision processing.

Stage 2 — Board Detection

OpenCV.js performs the following pipeline to locate the board within the screenshot:

- Convert RGBA image to grayscale
- Apply Gaussian blur (kernel 5×5) to reduce noise
- Canny edge detection to find sharp boundaries
- Hough line transform to detect the chessboard grid lines
- Find the largest square contour — that's the board
- Perspective transform to get a perfect top-down 640×640 view

Stage 3 — Piece Classification

The 640×640 board is split into 64 squares of 80×80 pixels each. A fine-tuned MobileNetV3 CNN (trained on 200k+ labeled piece images) classifies each square into one of 13 classes: empty + 6 piece types × 2 colors.

Classes	Training Data	Accuracy	Inference Time
13 (empty, ■■■■■■ × 2)	200,000 labeled squares	99.1% test accuracy	~35ms for full board (

Stage 4 — FEN Generation

The 8×8 piece grid is encoded into a FEN (Forsyth–Edwards Notation) string. The system also tracks castling rights (by watching king/rook positions across frames) and en-passant eligibility (by tracking pawn double-pushes).

Example: `rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1`



AI Engine & Move Analysis

Stockfish evaluation + Claude reasoning working together

Stockfish Integration (WebAssembly)

Stockfish 16 is compiled to WebAssembly and runs inside a Web Worker (so it never blocks the UI). The extension sends a UCI command with the FEN and receives the top-3 moves ranked by centipawn score.

UCI Command Flow:

```
uci ← Initialize engine
position fen ← Set board position
go depth 18 multipv 3 ← Analyze top 3 moves at depth 18
bestmove e2e4 ponder d7d5 ← Engine responds with best move
```

Claude API Integration

Each Stockfish result is enriched with a natural language explanation by calling Claude Sonnet. The prompt is carefully engineered to produce concise (2–3 sentence), jargon-free tactical explanations suitable for players of all skill levels.

Prompt Template:

```
SYSTEM: You are a chess coach. Given a board position and candidate moves,
explain each move in 2-3 sentences. Be concise. Use simple language.
Mention threats, tactical motifs, and positional goals.
```

USER:

```
Position (FEN): {fen}
Current player: {color}
Move history: {pgn_moves}
```

Engine top moves:

1. {move1} (eval: {eval1} centipawns)
2. {move2} (eval: {eval2} centipawns)
3. {move3} (eval: {eval3} centipawns)

Explain each move briefly.

Evaluation Score Guide

Centipawn Score	Meaning	UI Color	Label
+300 or more	Winning advantage	■ Green	Winning
+100 to +300	Clear advantage	■ Light Green	Advantage
+25 to +100	Slight edge	■ Blue	Slight Edge
-25 to +25	Equal position	■ White	Equal
-25 to -100	Slight disadvantage	■ Yellow	Slightly Worse
-100 to -300	Losing position	■ Orange	Losing
-300 or less	Heavily losing	■ Red	Critical

Overlay Design Principles

- Non-intrusive: The overlay sits outside the browser viewport and never covers the board
- Dark theme: Matches the dark aesthetic of Chess.com and Lichess night mode
- Keyboard shortcuts: Toggle overlay (Alt+C), next/prev suggestion (Alt+↑/↓)
- Draggable: Users can reposition the panel to any corner of the screen
- Minimal mode: Collapse to a small icon when you want full focus

Move Card UI Structure

Element	Description	Color
Move badge	Algebraic notation (e4, Nf3, O-O-O)	Accent green
Eval bar	Horizontal bar showing centipawn advantage	Green→Red gradient
Rank pill	BEST / GOOD / EQUAL labels	Green / Blue / Gray
Explanation text	Claude's 2–3 sentence natural language comment	White, 13px
Arrow overlay	Arrow drawn on board showing the suggested move	Semi-transparent green
Piece icon	SVG chess piece icon next to move name	White/Black based on turn

Settings Panel

- Analysis depth: 10–22 (slider, default 18)
- Number of moves shown: 1, 2, or 3
- Explanation verbosity: Brief / Detailed / Off
- Auto-hide when it's opponent's turn: Toggle
- Overlay position: Top-left / Top-right / Bottom-left / Bottom-right
- Hotkeys: Fully remappable
- Sound alerts: Beep when a blunder is detected



Post-Game Analysis Module

Full game review dashboard after each match

How It Works

After the game ends, the extension detects the result screen, extracts the full PGN, and opens the analysis dashboard in a new browser tab. The entire game is re-analyzed with Stockfish at depth 20 and every move is annotated by Claude.

Move Classification System

Symbol	Name	Definition	Threshold
!!	Brilliant	Only move that wins; unexpected sacrifice	Best move + eval jump > 200cp
!	Best	Objectively the best move in the position	Stockfish rank #1
	Good	One of the top moves, small eval difference	Within 50cp of best
?!	Inaccuracy	Slightly suboptimal; loses small advantage	50–100cp worse than best
?	Mistake	Significant error; changes game evaluation	100–300cp worse than best
??	Blunder	Game-losing error; drops major material	> 300cp worse than best

Dashboard Sections

	Accuracy Graph Line chart showing accuracy % over the course of the game for both players	HIGH
	Critical Moments Board positions at the 3 most impactful moves — where the game was won or lost	HIGH
	Opening Analysis Detected opening name, moves in theory, and where you deviated from book	HIGH
	Pattern Recognition Claude identifies recurring tactical themes: pins, forks, discovered attacks	HIGH
	AI Coaching Summary A paragraph summary from Claude acting as your coach, with 3 key takeaways	HIGH
	Replay Mode Step through every move with arrows, evaluation bar, and Claude commentary	HIGH



Development Roadmap

5 phases from MVP to full product launch

PHASE 1

Foundation

Weeks 1–3

PHASE 2

Vision Core

Weeks 4–6

PHASE 3

AI Engine

Weeks 7–9

PHASE 4

UI/UX Build

Weeks 10–12

PHASE 5

Launch

Weeks 13–16

Phase	Goals	Deliverables	Dependencies
1 Foundation (Wks 1–3)	- Project setup - Chrome ext boilerplate - CI/CD pipeline - Design system	- Working skeleton extension - GitHub repo + Actions - Component library	Node.js, Vite, React
2 Vision Core (Wks 4–6)	- OpenCV board detection - CNN piece classifier - FEN generation - Test suite	99%+ accurate board parser - Test dataset (500 positions) - Performance benchmarks	OpenCV.js, TensorFlow.js, training data
3 AI Engine (Wks 7–9)	- Stockfish WASM - UCI integration - Claude API calls - Caching layer	- Real-time move suggestions - Evaluation bar - Natural language explanations	Anthropic API key, Stockfish WASM
4 UI/UX Build (Wks 10–12)	- Overlay panel - Move cards UI - Post-game dashboard - Settings page	- Full UI feature complete - Keyboard shortcuts - Responsive overlay	Figma designs, React components
5 Launch (Wks 13–16)	- Beta testing - Bug fixes - Chrome Web Store - Electron packaging	- Public Chrome extension - Electron desktop app - v1.0 release	Chrome developer account, code signing

Risks & Mitigations

Technical, operational, and ethical risk planning

Risk	Probability	Impact	Mitigation Strategy
Chess site anti-cheat detects extension	MEDIUM	HIGH	Use passive screenshot approach (no DOM injection). Add 'tournament mode' that disables extension.
OpenCV board detection fails on custom board themes	LOW	MEDIUM	Support manual board region selection. Build theme detection into CNN. Ship with 50+ trained models.
Claude API rate limits exceeded during peak usage	LOW	MEDIUM	Cache common positions. Use Redis to avoid re-analyzing identical FEN strings. Implement exponential backoff.
Stockfish WASM too slow on older hardware	LOW	HIGH	Adaptive depth: reduce depth on low-end devices. Fall back to server-side Stockfish for critical moves.
Chrome Manifest V3 restrictions on background workers	LOW	MEDIUM	Use Service Workers per MV3 spec. Keep workers alive with alarms API. Test extensively on Chrome 110+.
User data privacy concerns (game data storage)	LOW	HIGH	All analysis runs locally by default. Cloud sync is opt-in. Zero game data sent to servers without explicit consent.

REST API Endpoints

Method	Endpoint	Description	Auth
POST	/api/analyze	Analyze FEN position, return top moves + eval	API Key
POST	/api/explain	Get Claude explanation for a specific move	API Key
POST	/api/games	Save completed game PGN to user account	JWT
GET	/api/games/{id}	Retrieve stored game with full analysis	JWT
GET	/api/games/{id}/moves	Get move-by-move analysis for a game	JWT
POST	/api/games/{id}/coach	Generate AI coaching summary for a game	JWT
GET	/api/user/stats	User performance stats, accuracy trends	JWT
DELETE	/api/games/{id}	Delete a saved game and its analysis	JWT

FEN Analysis Request Schema

```
{
  "fen": "rnbqkbnr/pppp1ppp/4p3/8/4P3/8/PPPP1PPP/RNBQKBNR w KQkq - 0 2",
  "depth": 18,
  "multipv": 3,
  "time_limit_ms": 150,
  "include_explanation": true,
  "player_elo": 1400,
  "game_phase": "opening"
}
```

Analysis Response Schema

```
{
  "fen": "rnbqkbnr/pppp1ppp/4p3/8/4P3/8/PPPP1PPP/RNBQKBNR w KQkq - 0 2",
  "evaluation": 0.32,
  "game_phase": "opening",
  "moves": [
    {
      "rank": 1, "uci": "d2d4", "san": "d4", "eval": 0.38,
      "label": "BEST",
      "explanation": "The Queen's Gambit setup. Controls the center with a second pawn and..."
    },
    {
      "rank": 2, "uci": "g1f3", "san": "Nf3", "eval": 0.31,
      "label": "GOOD",
      "explanation": "Developing the knight to a natural square, preparing castling..."
    }
  ],
  "latency_ms": 187
}
```

Chrome Extension (Primary Distribution)

- Build with Vite: `vite build --mode production` — outputs `dist/` folder
- Zip the `dist/` directory: `chess-ai-agent-v1.0.zip`
- Upload to Chrome Web Store Developer Dashboard
- Pass Chrome's automated review (typically 1–3 days)
- Publish — users install from `chrome.google.com/webstore`
- Updates pushed automatically when new version is published

Electron Desktop App (Advanced Users)

- Wraps the extension in a native desktop window with frameless overlay support
- Enables system-level screen capture (works with desktop chess apps, not just browsers)
- Built with `electron-builder` for cross-platform distribution
- macOS: DMG installer (`.dmg`) — signed with Apple Developer certificate
- Windows: NSIS installer (`.exe`) — signed with code signing certificate
- Linux: AppImage (`.AppImage`) — portable, no installation required
- Auto-updates via `electron-updater` pointing to GitHub Releases

Backend API Deployment

Environment	Platform	URL	Purpose
Development	Local Docker Compose	<code>localhost:3000</code>	Feature development
Staging	Railway.app	<code>staging-api.chessai.app</code>	QA + integration tests
Production	Railway.app + Vercel	<code>api.chessai.app</code>	Live users
CDN	Cloudflare	<code>assets.chessai.app</code>	Static assets + WASM files

Document End — Chess AI Agent Extension v1.0 Blueprint

All technologies, timelines, and specifications are subject to revision during development.

■ Built with Stockfish 16 · Claude Sonnet · OpenCV · React · Electron · TypeScript